IE1205 Digital Design:

L14: Synthesis of FSMs

Ahmed Hemani

School of EECS, KTH, Stockholm, Sweden

Synthesis of Arbitrary FSMs

FSM Design Steps

1. ***Analyze the problem***
2. ***Derive the State Diagram.***

Requires Problem Solving Skills

3. ***Symbolic State Transition Table***
4. ***Do State Assignment, i.e., for each symbolic state, assign a unique binary code to it.***
5. ***Use Karnaugh Map or similar method to derive next state and output logic***

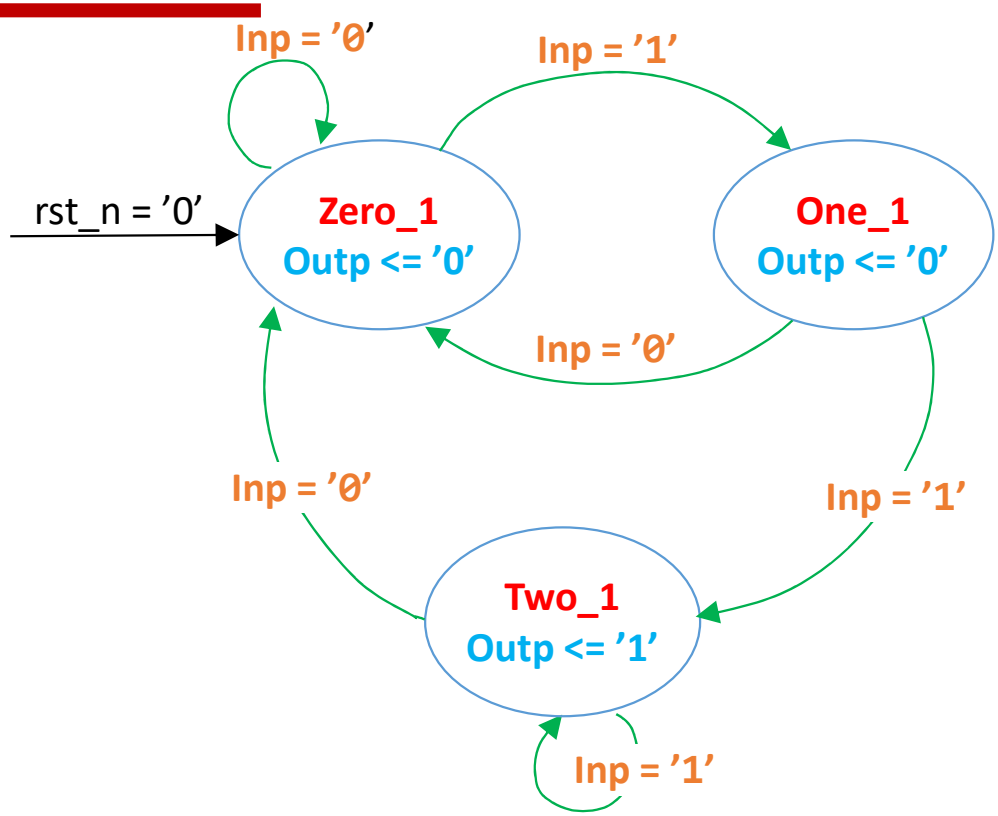
Mechanical

© Ahmed Hemani, KTH, Stockholm

Symbolic Moore State Table

State Machine Table is another way to represent FSMs

Input *inp*	**Present State**	**Next State**	**Output** *outp*
0	Zero_1	Zero_1	0
1	Zero_1	One_1	0
0	One_1	Zero_1	0
1	One_1	Two_1	0
0	Two_1	Zero_1	1
1	Two_1	Two_1	1

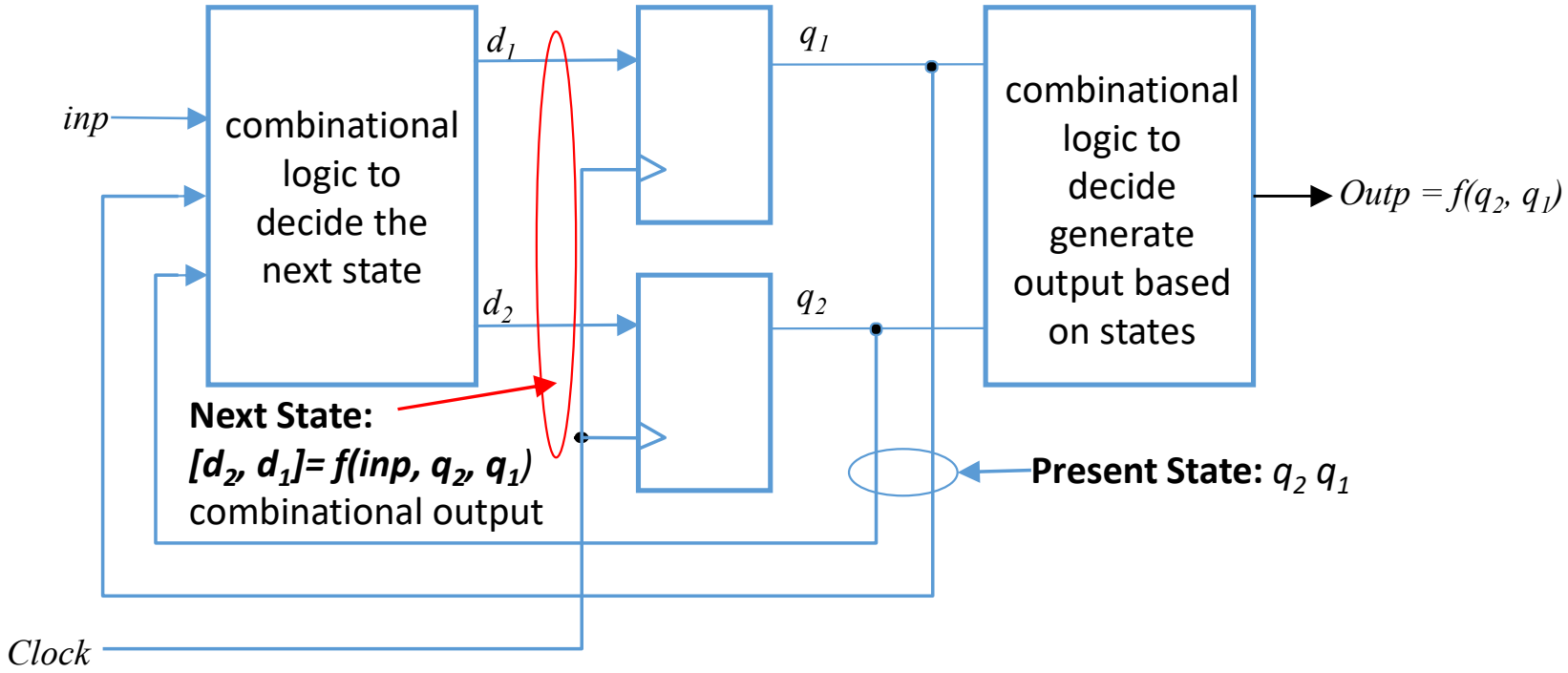


1. At present the state names are ***symbolic***.
2. In digital designs, everything has to be represented in terms of ones and zeros
3. The process of ***encoding states in terms of binary symbols*** (ones and zeros) is called ***state assignment***

© Ahmed Hemani, KTH, Stockholm

Moore Implementation of two in a row FSM

1. 3 states, Zero_1, One_1, Two_1, requires $\lceil \log_2(3) \rceil = 2$ flops
2. 2 flops can generate 4 states, three of which are legal



© Ahmed Hemani, KTH, Stockholm

State Assignment – binary code

1. The states were symbolic in the previous FSM table
2. **To implement the states digitally, we need to give them some ”binary names”**
3. For the moment, let us take some arbitrary decision to binary code the states:
   Zero_1 = ”00”, One_1= ”01”, Two_1=”10”
4. There exist strategies to assign states that is not arbitrary

Input *inp*	**Present State** q_2q_1	**Next State** d_2d_1	**Output** *outp*
0	00 (Zero_1)	00 (Zero_1)	0
1	00 (Zero_1)	01 (One_1)	0
0	01 (One_1)	00 (Zero_1)	0
1	01 (One_1)	10 (Two_1)	0
0	10 (Two_1)	00 (Zero_1)	1
1	10 (Two_1)	10 (Two_1)	1
-	11	--	-

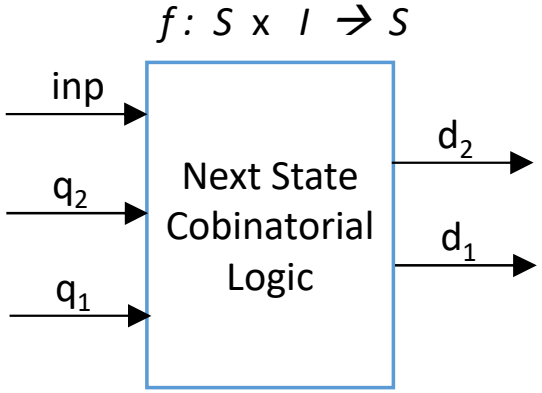
$d_2\ d_1 = f\ (q_2, q_1, inp)$

$Outp = f\ (q_2, q_1)$

1. Just like Boolean functions, FSMs are rarely fully specified. This results in don’t care conditions
2. Once again, like in Boolean Minimization, we can exploit the don’t care to minimize the next state and output logic

© Ahmed Hemani, KTH, Stockholm

Deriving the next state logic for d_2

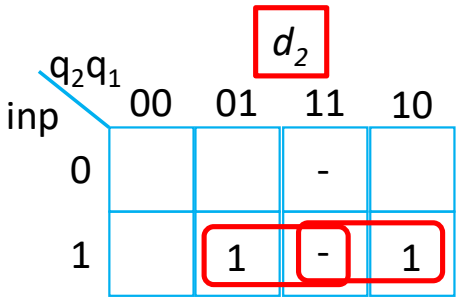


Inputs to Next State + Output Combinational Logic

Outputs of Next State + Output combinational Logic

Input *inp*	**Present State** q_2q_1	**Next State** d_2d_1	**Output** *outp*
0	0 0 (Zero_1)	0 0 (Zero_1)	0
1	0 0 (Zero_1)	0 1 (One_1)	0
0	0 1 (One_1)	0 0 (Zero_1)	0
1	0 1 (One_1)	**1** 0 (Two_1)	0
0	1 0 (Two_1)	0 0 (Zero_1)	1
1	1 0 (Two_1)	**1** 0 (Two_1)	1
-	1 1	--	-

1. There are four possible States
2. The fourth is illegal and cannot happen
3. It is treated as a don't care

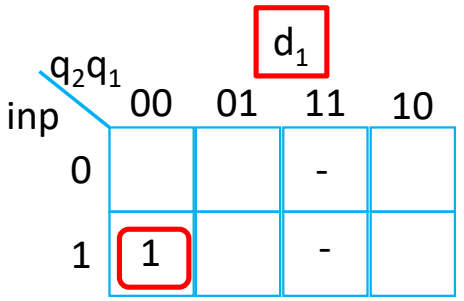


$d_2 = q_1inp + q_2inp$
$= inp(q_1 + q_2)$

© Ahmed Hemani, KTH, Stockholm

Deriving the next state logic for d_1

Input *inp*	Present State q_2q_1	Next State d_2d_1	Output *outp*
0	0 0 (Zero_1)	0 0 (Zero_1)	0
1	0 0 (Zero_1)	0 **1** (One_1)	0
0	0 1 (One_1)	0 0 (Zero_1)	0
1	0 1 (One_1)	1 0 (Two_1)	0
0	1 0 (Two_1)	0 0 (Zero_1)	1
1	1 0 (Two_1)	1 0 (Two_1)	1
-	1 1	--	-

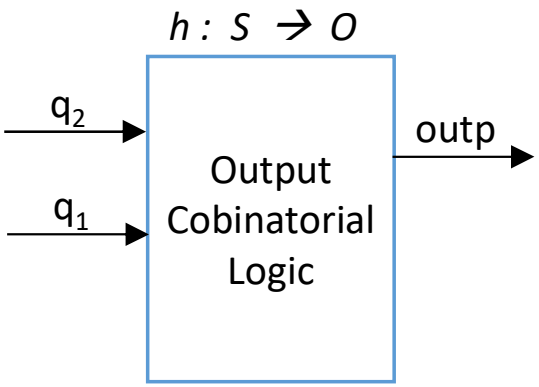


$d_1 = q_2'q_1'inp$

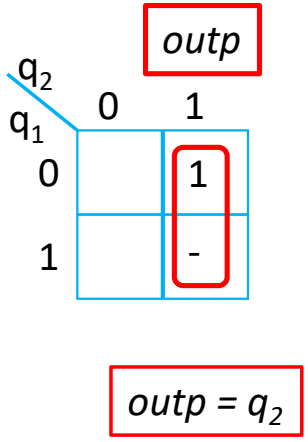
$$d_2 \boxed{d_1} = f(q_2\ q_1\ inp)$$

© Ahmed Hemani, KTH, Stockholm

Deriving the output logic for outp



Input *inp*	Present State q_2q_1	Next State d_2d_1	Output *outp*
0	0 0 (Zero_1)	0 0 (Zero_1)	0
1	0 0 (Zero_1)	0 1 (One_1)	0
0	0 1 (One_1)	0 0 (Zero_1)	0
1	0 1 (One_1)	1 0 (Two_1)	0
0	1 0 (Two_1)	0 0 (Zero_1)	**1**
1	1 0 (Two_1)	1 0 (Two_1)	**1**
-	1 1	- -	-



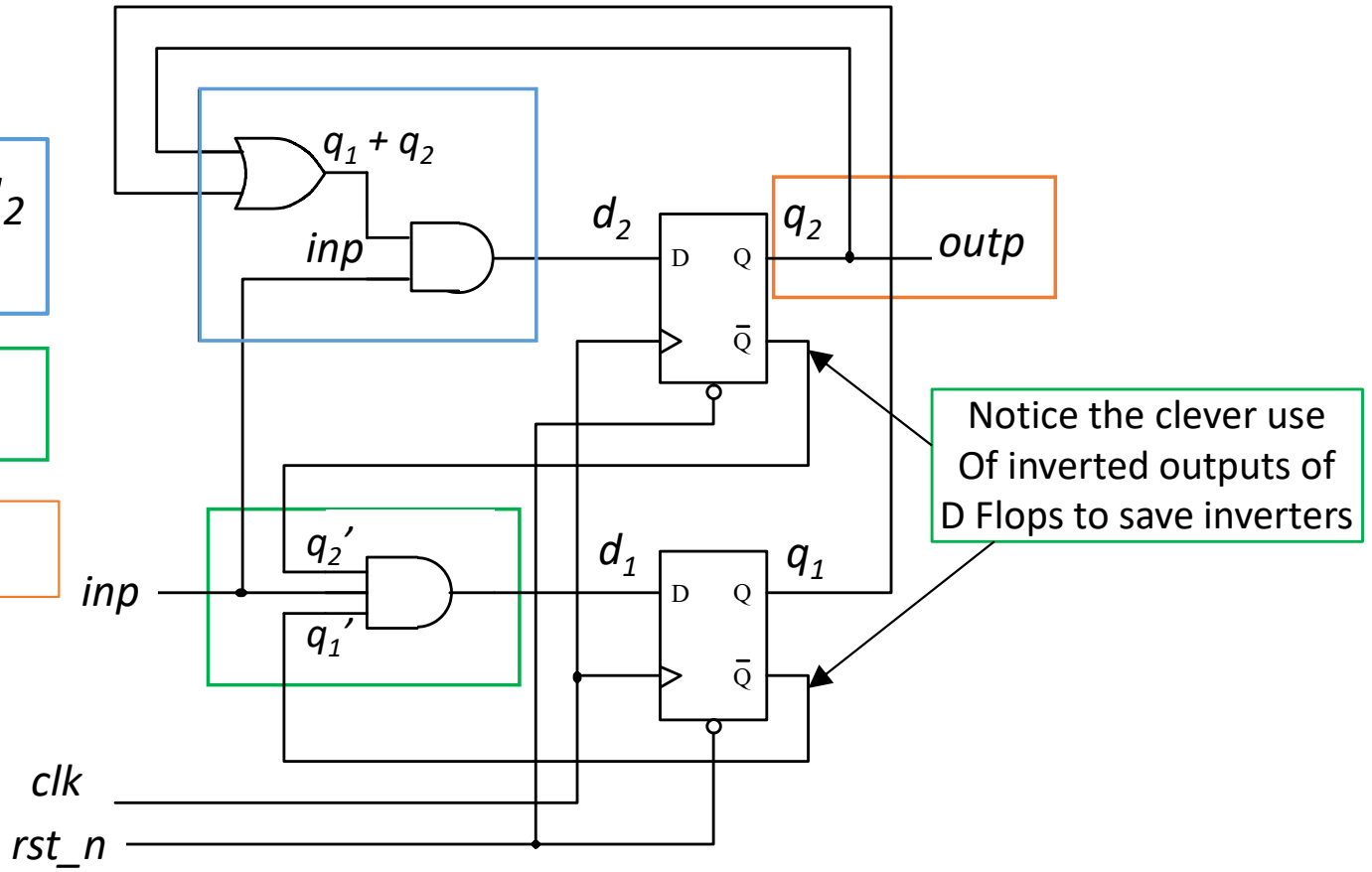
© Ahmed Hemani, KTH, Stockholm

Logic Implementation of Next State and Output Functions

$d_2 = inp\ q_1 + inp\ q_2$
$= inp\ (q_1 + q_2)$

$d_1 = inp\ q_1' q_2'$

$outp = q_2$



© Ahmed Hemani, KTH, Stockholm

```
module two_in_a_row_moore_enc_1(
  input  logic inp,
  input  logic clk,
  input  logic rst_n,
  output logic outp
);

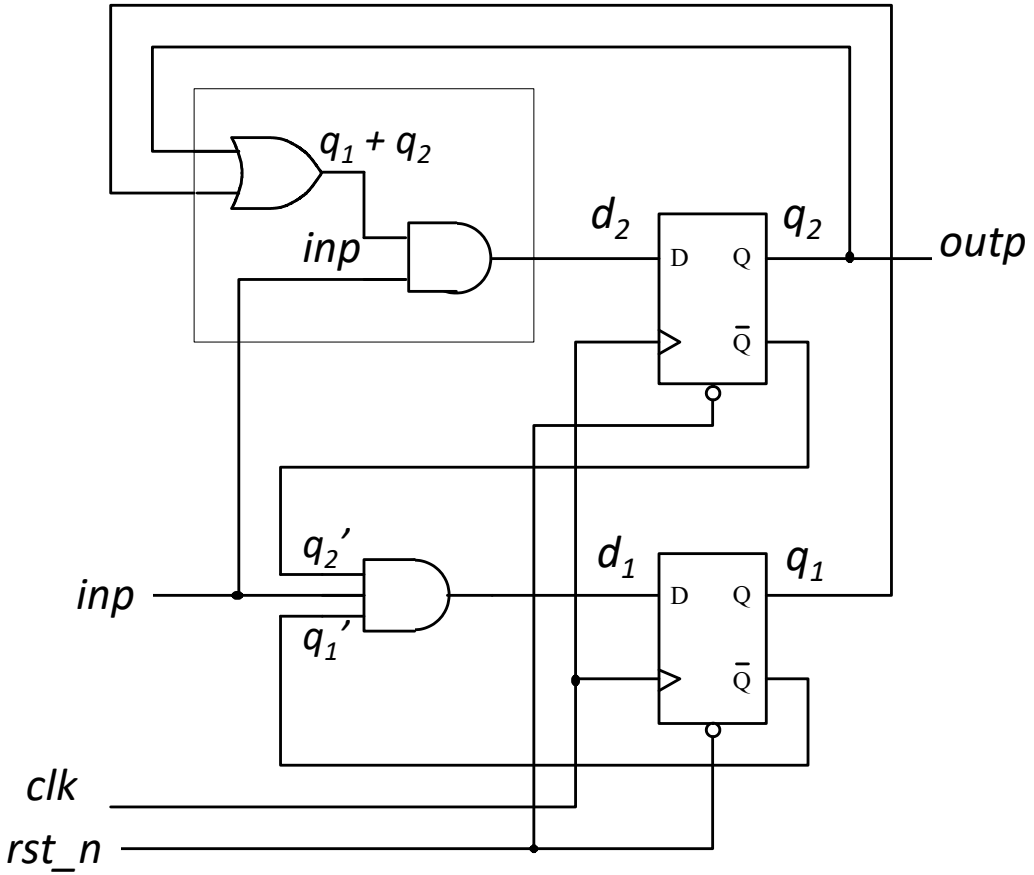
  // Define state type
  typedef enum logic [1:0] {
    Zero_1 = 2'b00,
    One_1  = 2'b01,
    Two_1  = 2'b10
  } state_ty;

  state_ty pres_state, next_state;

  // 1. State Register Process (Sequential Logic)
  always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n)
      pres_state <= Zero_1;
    else
      pres_state <= next_state;
  end

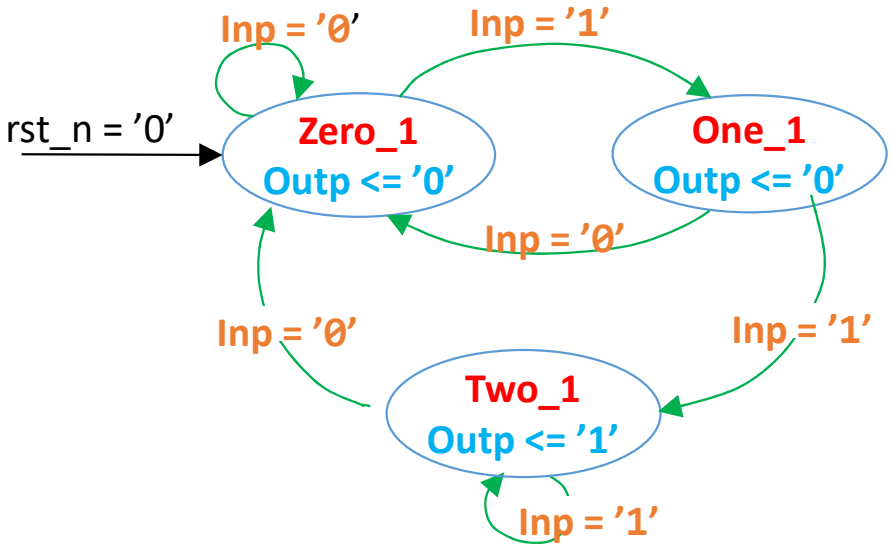
  // 2. Combinational Logic for Next-State and Output Logic
  always_comb begin
     next_state[1] = (pres_state[0] | pres_state[1]) & inp;
     next_state[0] = !pres_state[0] & !pres_state[1] & inp;
     outp = pres_state[1];
  end

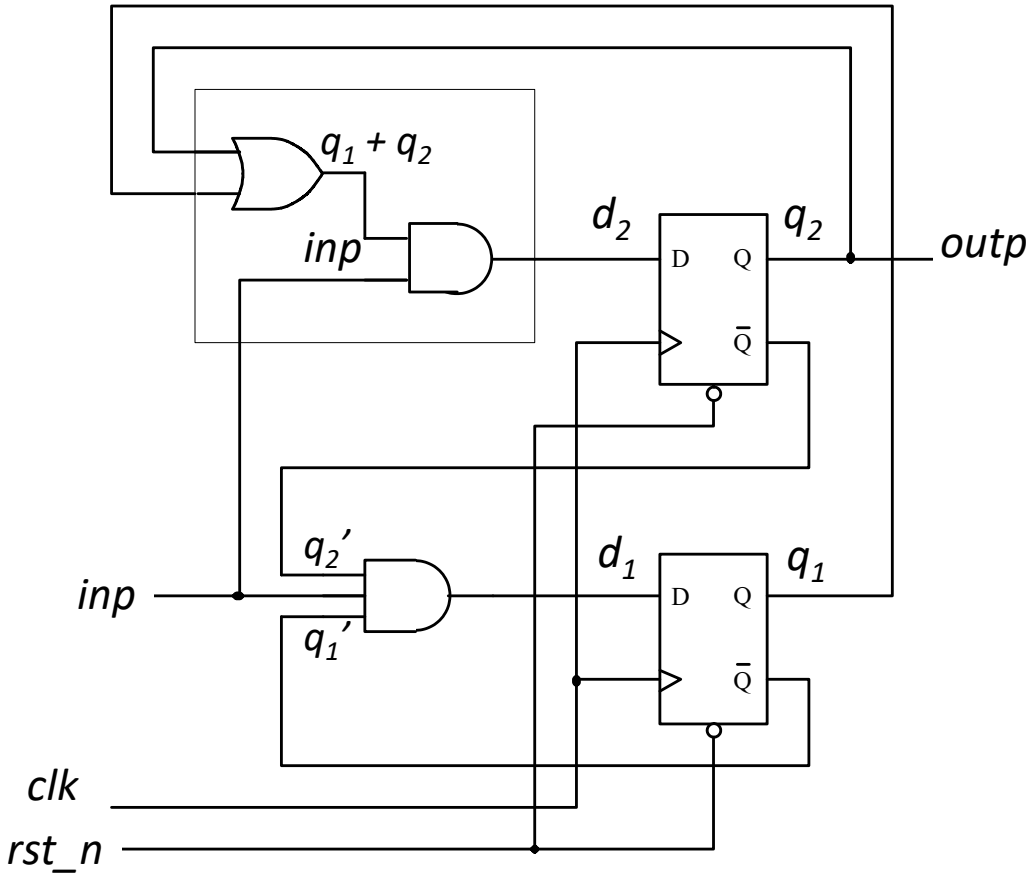
endmodule
```



© Ahmed Hemani, KTH, Stockholm

Quiz Time



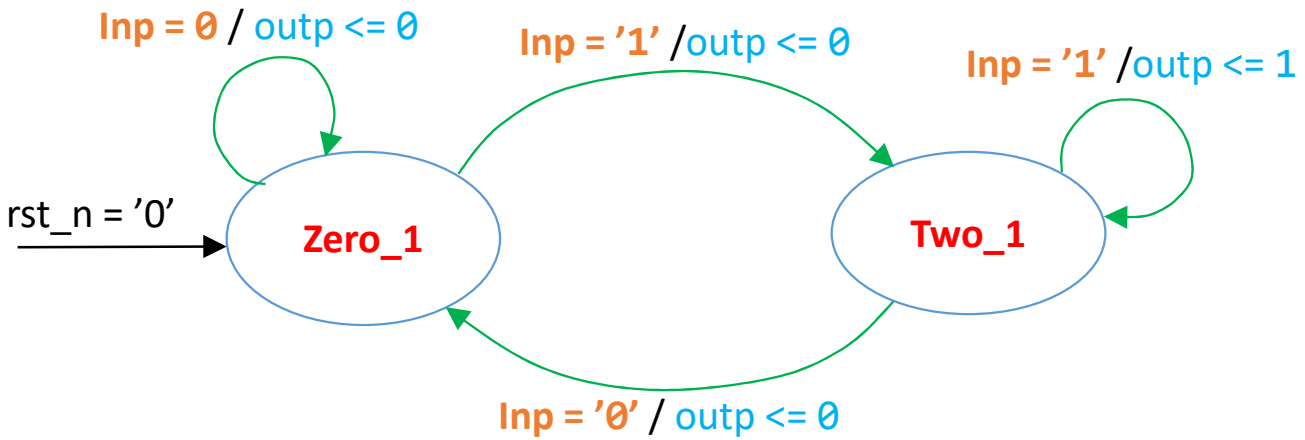


Which of the two diagrams: State Diagram and Logic Schematic is easier to understand what is the functionality?

A: State Diagram
B: Logic Schematic
C: SystemVerilog Code
E: All three are equally easy
F: All three are equally hard

© Ahmed Hemani, KTH, Stockholm

Synthesis of Mealy FSM for two-in-a-row Example



Input inp	Present State	Next State	Output q
0	Zero_1	Zero_1	0
1	Zero_1	Two_1	0
0	Two_1	Zero_1	0
1	Two_1	Two_1	1

Notice that unlike Moore state table, in the Mealy state table, the output changes with inputs even in the same present state

There are only two states and we need only one flop

© Ahmed Hemani, KTH, Stockholm

State Assignment & Synthesis of Next State and Output Logic

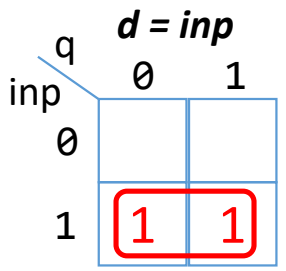
1. *Since there are two States, we just need one Flop*
2. *Let us Assign*
   a. *Code 0 to the state Zero_1*
   b. *Code 1 to the state Two_1*

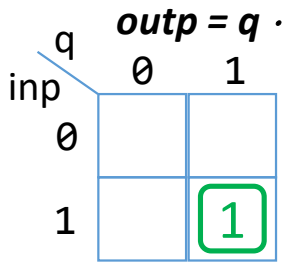
inputs		Outputs	
Input inp	**Present State q**	**Next State d**	**Output outp**
0	0 (Zero_1)	0 (Zero_1)	0
1	0 (Zero_1)	1 (Two_1)	0
0	1 (Two_1)	0 (Zero_1)	0
1	1 (Two_1)	1 (Two_1)	1

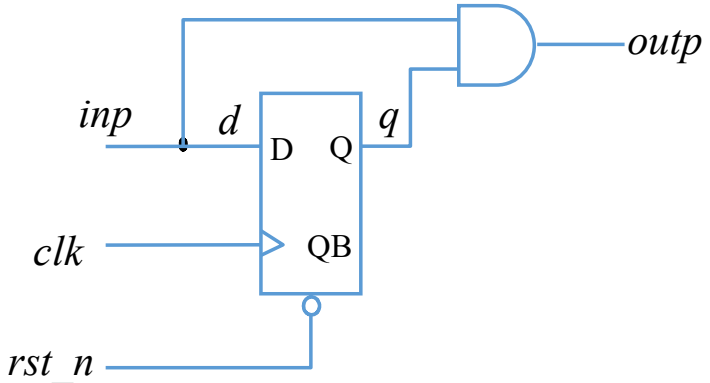
Logic for next state is trivial.

When input is 0, the next state is Zero_1("0")

When input is 1, the next state is Two_1 ("1")



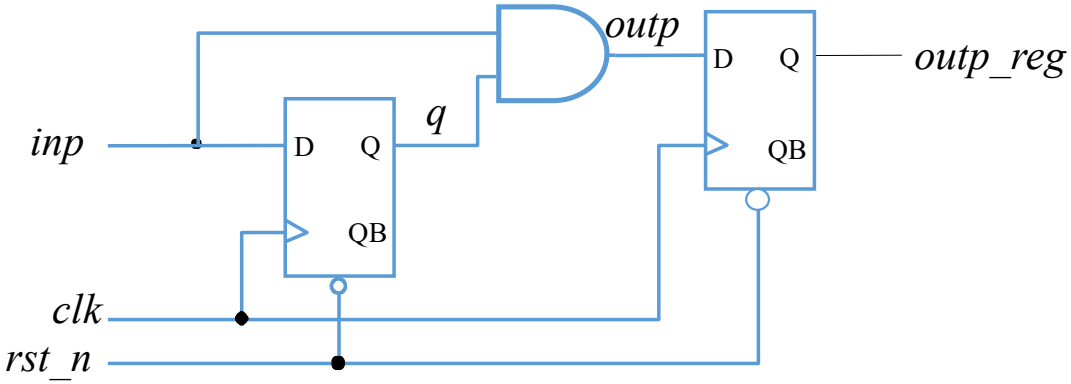




© Ahmed Hemani, KTH, Stockholm

Mealy FSM with registered Outputs

1. *The disadvantage with the Mealy machine is that its output can change during the clock cycle*
2. *To avoid changing the output during the clock period, register the output as follows.*
3. *This is a commonly adopted practice*



© Ahmed Hemani, KTH, Stockholm

```
module two_in_a_row_mealy_enc(
  input  logic inp,
  input  logic clk,
  input  logic rst_n,
  output logic outp
);

  // Define state type
  typedef enum logic {
    Zero_1 = 1'b0,
    Two_1  = 1'b1
  } state_ty;

  state_ty pres_state, next_state;

  // 1. State Register Process (Sequential Logic)
  always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n)
      pres_state <= Zero_1;
    else
      pres_state <= next_state;
  end

  // 2. Combinational Logic for Next-State and Output Logic
  always_comb begin
     next_state = state_ty'(inp);
     outp = inp & pres_state;
  end

endmodule
```

```
module two_in_a_row_mealy_enc_reg(input  logic inp,
                                  input  logic clk,
                                  input  logic rst_n,
                                  output logic outp);

   // Define state type
   typedef enum logic {Zero_1 = 1'b0,
                       Two_1  = 1'b1} state_ty;

   state_ty pres_state, next_state;
   logic outp_intern;

   // 1. State Register Process (Sequential Logic)
   always_ff @(posedge clk or negedge rst_n) begin
      if (!rst_n) begin
            pres_state <= Zero_1;
            outp <= 0;
      end
      else begin
            pres_state <= next_state;
            outp <= outp_intern;
      end
   end

   // 2. Combinational Logic for Next-State and Output Logic
   always_comb begin
      next_state = state_ty'(inp);
      outp_intern = inp & pres_state;
   end

endmodule
```


© Ahmed Hemani, KTH, Stockholm

Quiz Time

Typically, which type of FSM will have less number of states
A: Mealy machine will less number of states
B: Moore machine will have less number of states
C: It makes no difference, depending on the problem one or the other type of FSM can have more states

Which machine, Registered Mealy or Moore, takes more cycles to get the same sequential function done
A: Mealy machine
B: Moore machine
C: Depends on the problem
D: Both will take the same number of cycles

© Ahmed Hemani, KTH, Stockholm

Incompletely Specified FSMs Reliability Issues

Incompletely specified FSMs deserve care

1. *N Flops make 2^N states possible*

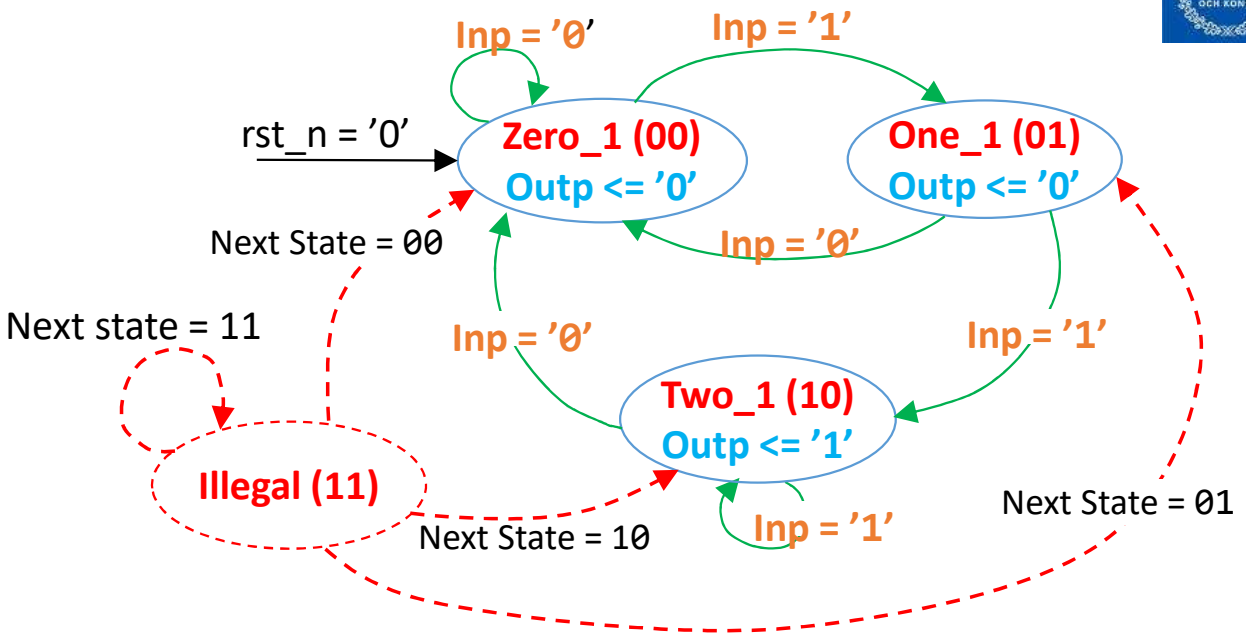
2. *Number of legal states is M*
   $M < 2^N$. Number of illegal states = $(2^N - M)$
3. *Noise and other hardware issues can cause an FSM to land in an illegal states*

4. *To improve reliability, one can assign the start state or some legal state as the next state from illegal states, rather than treating it as a don't care to save gates*

5. *If an FSM is stuck in an illegal state with no transition to start or some legal state, the only way to make the FSM operational again is to assert synchronous or asynchronous reset*

© Ahmed Hemani, KTH, Stockholm

Input *inp*	Present State q_2q_1	Next State d_2d_1	Output *outp*
0 1	00 (Zero_1) 00 (Zero_1)	00 (Zero_1) 01 (One_1)	0 0
0 1	01 (One_1) 01 (One_1)	00 (Zero_1) 10 (Two_1)	0 0
0 1	10 (Two_1) 10 (Two_1)	00 (Zero_1) 10 (Two_1)	1 1
-	11	--	-



1. Treating Next State as 11 will improve our potential to minimize Next State Logic
   More 1s, the greater is the probablity of enlarging encirclements – neighbourhood – in Karnaugh Map
2. However, if use 11 as the next state we are designing a logic that will remain stuck in the illegal 11 state if it accidently lands in the illegal state.
3. It is better to assign the next state as
   a. `00` (no potential for minimization) or
   b. `10` or `01`. This will retain some some potential to enhance minimization possibility
4. The outputs should also be carefully assigned, so that if the FSM accidently lands in an illegal state, the output is a safe value. Not all outputs may be critical.

© Ahmed Hemani, KTH, Stockholm